

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

17 - Procedures

Introduction

- A procedure or function is one tool programmers use to structure programs
 - to make them easier to understand
 - to allow code to be reused
- Procedures allow programmer to concentrate on just one portion of the task at a time
 - Parameters act as an interface between procedure and rest of program and data, since they can pass values and return results
- Procedures are one way to implement abstraction in SW

Procedure Execution

- Program must follow six steps
 1. Put parameters in place where procedure can access them
 2. Transfer control to procedure
 3. Acquire the storage resources needed for procedure
 4. Perform the desired task
 5. Put result value in a place where calling program can access it
 6. Return control to the point of origin

Registers Allocation

- As mentioned previously, registers are the fastest place to hold data in a computer
 - Use them as much as possible

Registers Allocation

- MIPS SW follows the following convention for procedure calling in allocating its 32 registers:
 - `$a0–$a3`: four argument registers to pass parameters
 - `$v0–$v1`: two value registers to return values
 - `$ra`: one return address register to return to point of origin

Jump-and-link Instruction

- MIPS assembly language includes instruction for procedures
- It jumps to an address and simultaneously saves the address of the following instruction in register \$ra
- **jump-and-link** instruction (**j_al**)
 - Written: **j_al ProcedureAddress**

Jump-and-link Instruction

- The *link* portion of the name means that an address or link is formed that points to the calling site to allow the procedure to return to the proper address
- This “link,” stored in register `$ra` (31) is called **return address**
 - Needed because same procedure could be called from several parts of the program

Jump-register Instruction

- MIPS use **jump register** instruction (**jr**), meaning an unconditional jump to the address specified in a register:

jr \$ra

- It jumps to address stored in register \$ra

Procedure Calling

- Calling program, or **caller**, puts the parameter values in \$a0–\$a3 and uses **jal X** to jump to procedure X (named **callee**)
- Callee then performs the calculations, places the results in \$v0 and \$v1, and returns control to the caller using **jr \$ra**

Program Counter

- A register holds address of current instruction being executed
 - For historical reasons, this register is called **program counter**, (*PC*)
 - more sensible name would have been *instruction address register*
- **jal** instruction saves $PC + 4$ in register $\$ra$ to link to the following instruction to set up procedure return

Using More Registers

- When compiler needs more registers for procedure than the four argument and two return value registers
- Since we must cover our tracks after our mission is complete
 - Any registers needed by caller must be **restored** to values that they contained *before* the procedure was invoked
 - Example of need to **spill registers** to memory
- Ideal data structure for spilling registers is **stack**
 - Last-In-First-Out queue

Stack

- Needs **pointer** to most recently allocated address in stack to show where next procedure should place registers to be spilled or where old register values are found
 - **Stack pointer** adjusted by one word for each register that is saved or restored
 - MIPS reserves register 29 for stack pointer, **\$sp**

Stack Operations

- Transferring data to and from the stack
 - placing data onto stack is called a **push**
 - removing data from stack is called a **pop**
- Stacks grow from **higher** addresses to **lower** addresses
 - push values onto stack by subtracting from SP: $\$sp = \$sp - 4$
 - pop values from stack by adding to SP: $\$sp = \$sp + 4$
 - Adding to SP shrinks stack, thereby popping values off stack

Example

- C/Java procedure:

```
int proc_example (int g, h, i, j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

What is the compiled MIPS assembly code?

Example (cont.)

- Parameter variables
 - g, h, i, and j correspond to argument registers \$a0, \$a1, \$a2, and \$a3,
 - f corresponds to \$s0

```
int proc_example (int g, h, i, j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

Example (cont.)

- Next, save registers used by procedure
 - Save three registers: \$s0, \$t0, and \$t1
 - Push old values onto stack
 - Creating space for three words (12 bytes) on stack and then store them

```
int proc_example (int g, h, i, j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

```
addi $sp, $sp, -12 # adjust stack for 3 items
sw $t1, 8($sp) # save $t1 for use afterwards
sw $t0, 4($sp) # save $t0 for use afterwards
sw $s0, 0($sp) # save $s0 for use afterwards
```


Example (cont.)

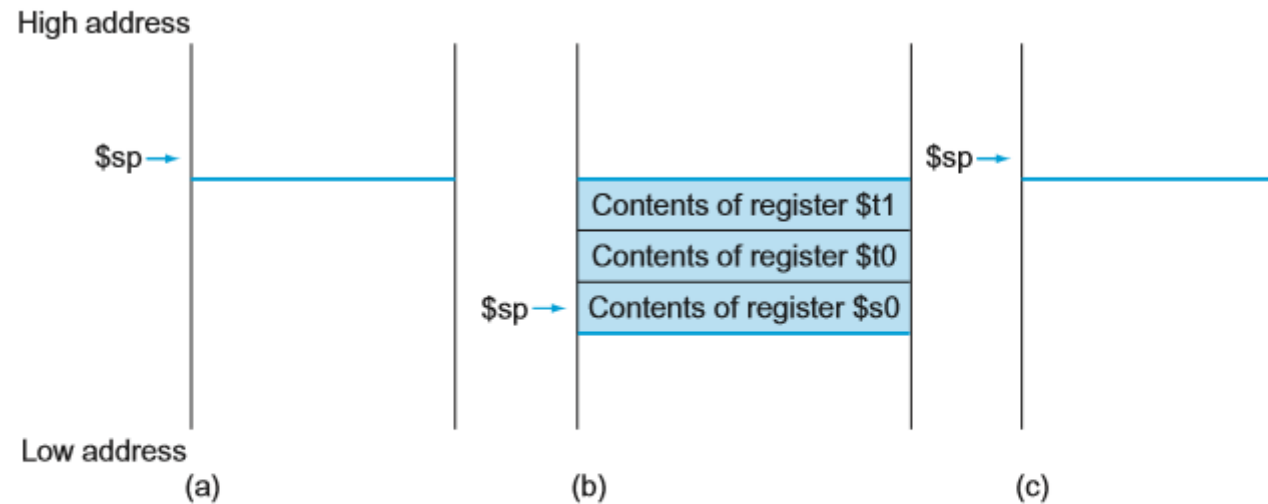


FIGURE 2.10 The values of the stack pointer and the stack (a) before, (b) during, and (c) after the procedure call. The stack pointer always points to the “top” of the stack, or the last word in the stack in this drawing.

Example (cont.)

- Next three statements correspond to body of procedure

```
add $t0,$a0,$a1 # register $t0 contains g + h
add $t1,$a2,$a3 # register $t1 contains i + j
sub $s0,$t0,$t1 # f=$t0-$t1, i.e. (g+h)-(i+j)
```

- To return value of `f`, copy it into a return value register

```
add $v0,$s0,$zero # returns f ($v0 = $s0 + 0)
```

Example (cont.)

- Before returning, restore three old values of registers saved by “popping” them from stack:

```
lw $s0, 0($sp) # restore $s0 for caller
lw $t0, 4($sp) # restore $t0 for caller
lw $t1, 8($sp) # restore $t1 for caller
addi $sp,$sp,12 # adjust stack to delete 3 items
```

- Procedure ends with a jump register using the return address:

```
jr $ra # jump back to calling routine
```

Example (cont.)

- Java code:

```
int proc_example (int g, h, i, j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

MIPS code:

proc_example:

```
addi $sp, $sp, -12
sw   $t1, 8($sp)
sw   $t0, 4($sp)
sw   $s0, 0($sp)
add  $t0, $a0, $a1
add  $t1, $a2, $a3
sub  $s0, $t0, $t1
add  $v0, $s0, $zero
lw   $s0, 0($sp)
lw   $t0, 4($sp)
lw   $t1, 8($sp)
addi $sp, $sp, 12
jr   $ra
```

Preserved Registers

- We used temporary registers and assumed their old values must be saved and restored
- To avoid saving and restoring a register whose value is never used, which might happen with a temporary register, MIPS separates 18 of registers into two groups
 - \$t0–\$t9: temporary registers that are not preserved by callee on a procedure call
 - \$s0–\$s7: saved registers that must be preserved on a procedure call (if used, callee saves and restores them)

Preserved Registers

- Reduces register spilling
- In the previous example, since caller does not expect registers \$t0 and \$t1 to be preserved across procedure call, we can drop two stores and two loads from code
- We still must save and restore \$s0, since callee must assume that the caller needs its value

Preserved Registers

- Several schemes preserve stack, guaranteeing that caller will get same data back on a load from stack as it stored onto stack
- Stack above `$sp` is preserved by making sure callee does not write above `$sp`
- `$sp` is itself preserved by callee adding exactly same amount that was subtracted from it
- Other registers are preserved by saving them on the stack (if they are used) and restoring them from there

Preserved Registers

Preserved	Not preserved
Saved registers: \$s0-\$s7	Temporary registers: \$t0-\$t9
Stack pointer register: \$sp	Argument registers: \$a0-\$a3
Return address register: \$ra	Return value registers: \$v0-\$v1
Stack above the stack pointer	Stack below the stack pointer